LiquidHaskell: Experience with Refinement Types in the Real World

Niki Vazou, Eric L. Seidel, Ranjit Jhala

Szabó Márk

Haskell Background

Haskell's Type System

Haskell is a **statically typed** language: every expression has a type, and types are checked before the program runs.

Some common built-in types:

```
Int       -- integer
Float     -- floating point number
Double    -- floating point number
Num       -- [Int, Float, Double, ...]
Bool      -- True or False
Char      -- single character
String    -- text
```

Type Annotations and Inference

The operator :: means “has type”. Annotations are optional: Haskell can **infer** most types.

Annotated:

```
age :: Int
age = 23

name :: String
name = "Mark"
```

Inferred, inspected in GHCi:

```
double x = x * 2

:t True
True :: Bool

:t double
double :: Num a => a -> a
```

Function Types

The arrow (→) separates input and output types.

```
add :: Int -> Int -> Int
add x y = x + y
```

Every function takes **one argument**: `add` takes `x` and returns a function that takes `y`.

A C-like version gives the **same result**, but is not logically the same, since it takes both arguments at once:

```
int add(int x, int y) { return x + y; }
```

Limits of The Type System

We can’t define constraints for the types of a function like:

- Conditions about the value: `x > 0`
- Relationships between values: `a > b`

To check that the input is non-negative, we need a dynamic test:

```haskell
sqrtRoot :: Double -> Double
sqrtRoot x
  | x >= 0    = sqrt x
  | otherwise = error "x must be non-negative"
```

Refinement Types and LiquidHaskell

LiquidHaskell

Many of these properties and relationships can be verified using a combination of **Refinement Types** and **external SMT solvers**.

This paper introduces and experiments with **LiquidHaskell** a refinement typechecker used to verify more than **10,000** lines of Haskell code across popular libraries.

Including: *bytestring*, *text*, *vector-algorithms* and *xmonad*.

Optional checker

Refinement types are written inside `{-@ ... @-}` comments, so normal Haskell compilers ignore them. Verification happens before the program runs and adds no runtime checks.

Refinement Types

Refinement types:

- contain a **predicate** which is assumed to be true for any element of the refined type.
- can express **preconditions** when used as function arguments or **postconditions** when used as return types.

Examples:

positive natural numbers

$x : \{n \in \mathbb{N} \mid n > 0\}$

result is a positive number

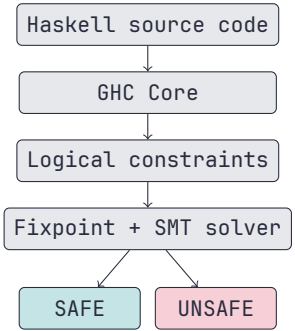
$f : \mathbb{N} \to \{y \in \mathbb{N} \mid y > 0\}$

result > input

$f : (x : \mathbb{N}) \to \{y \in \mathbb{N} \mid y > x\}$

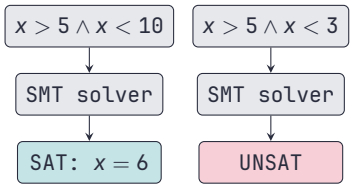
LiquidHaskell Architecture

- **Core:** GHC translates Haskell into a simpler intermediate representation; source locations are kept for error reporting.
- **Constraints:** refinement types become logical constraints that must hold.
- **Solution:** a fixpoint algorithm and an SMT solver (e.g. *Z3*, *CVC4*) check whether the constraints can be satisfied.
- **Result:** **SAFE**, or **UNSAFE** with errors mapped back to source locations.



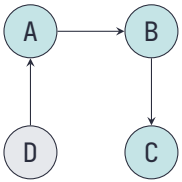
SMT Solvers

- An SMT solver takes a logical formula and answers whether values exist that make it true.
- **SAT**: yes, and it gives an example.
- **UNSAT**: no such values exist.
- LiquidHaskell: if the constraints are satisfiable, the program is **SAFE**.



Fixpoints

- A **fixpoint** of f is a value that f does not change: $f(x) = x$.
- To find one, apply f again and again until the result stays the same.



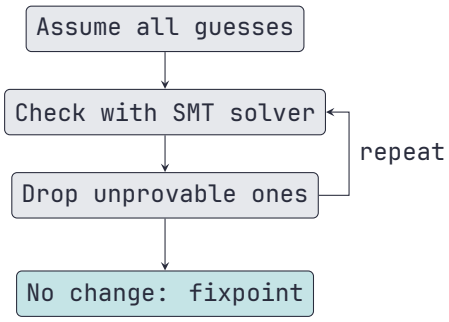
Example: which nodes can we reach from A?
Each step adds the next nodes along the arrows.



no change: fixpoint

Fixpoint in LiquidHaskell: Inferring Refinements

- The user writes only some refinements. For helper functions and local values LiquidHaskell has to **infer** them.
- Guesses come from the user's refinements and unprovable ones are dropped.
- Recursion links the checks, so this repeats until nothing changes.



LiquidHaskell Basics

```
1  {-@ type Natural = {v:Int | v >= 0} @-}
2  {-@ type Pos = {v:Int | v > 0} @-}
3
4  {-@ increment :: Natural -> Pos @-}
5  increment :: Int -> Int
6  increment x = x + 1
7
8  good1 = increment 0
9  good2 = increment 4
10
11 -- verification will fail here stating: Type Mismatch
12 bad = increment (-1)
```

LiquidHaskell Basics

```
1  {-@ type Natural = {v:Int | v >= 0} @-}
2  {-@ type Pos = {v:Int | v > 0} @-}
3
4  {-@ divide :: n:Natural -> d:Pos -> {v:Natural | v <= n} @-}
5  divide :: Int -> Int -> Int
6  divide x y = x `div` y
7
8  good = divide 5 2
9
10 -- With LH -> Type Mismatch before executing the code
11 -- Without LH -> runtime error: divide by zero
12 bad = divide 5 0
```

LiquidHaskell Parameterized Types

Value parameters are written in uppercase: here `Low` and `High`.

```
{-@ type Rng Low High = {v:Int | v >= Low && v <= High} @-}
{-@ grade :: Rng 0 100 -> Int @-}
grade :: Int -> Int
grade score
    | score >= 90 = 5
    | score >= 75 = 4
    | score >= 60 = 3
    | score >= 50 = 2
    | otherwise   = 1
good = grade 85
bad = grade 120
```

LiquidHaskell Type Aliases

Type Aliases: used for reusability and readability.

```
1  -- with type alias
2  {-@ predicate Between Low High V = V >= Low && V <= High @-}
3  {-@ type Rng Low High = {v:Int | Between Low High v} @-}
4
5  -- we can use the predicate inline as well
6  {-@ grade :: Rng 0 100 -> {v:Int | Between 1 2 v} @-}
7  grade :: Int -> Int
8  grade score
9      | score >= 50 = 2
10     | otherwise   = 1
11
12 good = grade 85
13 bad = grade 120
```

Refining Data and Interfaces

LiquidHaskell Measures

So far, the specifications have been limited to comparisons and arithmetic operations on **primitive** values. **Measures** describe **inductive properties** of algebraic data types.

```
1 {-@ measure myLen @-}
2 myLen :: [a] -> Int
3 myLen []     = 0
4 myLen (x:xs) = 1 + myLen xs
```

- One equation for each data constructor: `[]` and `(:)`.
- Each right-hand side must belong to the restricted refinement logic.
- `measure myLen` makes this Haskell function available in refinements.

LiquidHaskell Measures: Minimum Length

```
1 {-@ type MinLength a N = {v:[a] | myLen v >= N} @-}
2
3 {-@ good :: MinLength Int 3 @-}
4 good :: [Int]
5 good = [1, 2, 3, 4]
6
7 {-@ bad :: MinLength Int 3 @-}
8 bad :: [Int]
9 bad = [1, 2]
```

- `a` is the element type; `N` is the minimum length.
- **SAFE:** `good` has length 4, and $4 \geq 3$.
- **UNSAFE:** `bad` has length 2, and $2 < 3$.

LiquidHaskell Measures: Constructor Refinements

Each list constructor gives LiquidHaskell a **length fact**:

```
[]  :: {v:[a] | myLen v == 0}
(:) :: x:a -> xs:[a] -> {v:[a] | myLen v == 1 + myLen xs}
```

These facts prove that adding an element increases the minimum length:

```
{-@ prepend :: a -> MinLength a 2 -> MinLength a 3 @-}
prepend x xs = x : xs

good = prepend 1 [2, 3]
bad = prepend 1 [2]
```

$myLen\ xs \geq 2$, so $myLen\ (x : xs) = 1 + myLen\ xs \geq 3$.

LiquidHaskell Refined Data Types

Every value of a data type may need to satisfy an **invariant**.

```
1 data CSV a = CSV { cols :: [String], rows :: [[a]] }
```

- `cols`: the column names in the header.
- `rows`: the table's rows, containing values of type `a`.
- Haskell allows rows of different lengths.

CSV invariant

Every row must have exactly as many entries as the header has column names.

LiquidHaskell Refined Data Types: Definition

A field's refinement can refer to an **earlier field**.

```
1 {-@ type ListL a X = {v:[a] | len v == len X} @-}
2
3 {-@ data CSV a = CSV { cols :: [String]
4                     , rows :: [ListL a cols] } @-}
```

- `ListL a X`: a list with the same length as `X`.
- `rows`: a list of rows, each matching the length of `cols`.
- `len` is LiquidHaskell's built-in list length measure.

LiquidHaskell Refined Data Types: Verification

```
1 good = CSV ["Month", "Days"]
2              [ ["Jan", "31"]
3              , ["Feb", "28"]
4              , ["Mar", "31"] ]
5
6 bad =  CSV ["Month", "Days"]
7             [ ["Jan", "31"]
8             , ["Feb", "28"]
9             , ["Mar"] ]
```

LiquidHaskell Refined Type Classes

A **type class** defines a shared interface. Refinements give the method a **shared contract**:

```
{-@ class Reward a where
      reward :: a -> points:{Int | points >= 0}
             -> {v:Int | v >= points}
@-}
```

- **Precondition:** input points must be non-negative.
- **Postcondition:** the result cannot be smaller than the input.
- Every instance must satisfy the **same contract**.

LiquidHaskell Refined Type Classes: Bool Instance

A Boolean flag decides whether to add a bonus.

```
1 instance Reward Bool where
2   reward bonus points
3     | bonus     = points + 10
4     | otherwise = points
```

- `True`: add 10 points.
- `False`: keep the original points.
- Both branches satisfy $v \geq \mathit{points}$.

LiquidHaskell checks the instance against the class contract.

LiquidHaskell Refined Type Classes: Verification

Callers must meet the precondition:

```
1 good = reward True 20       -- returns 30
2 bad = reward True (-1)      -- violates points >= 0
```

Instances must prove the postcondition. A deduction breaks it:

```
1 instance Reward Bool where
2   reward bonus points
3     | bonus     = points - 10     -- violates v >= points
4     | otherwise = points
```

Termination

LiquidHaskell Termination

Haskell only evaluates expressions when their results are needed (*non-strict evaluation*).

- Some functions may run forever without producing a result.
- **LiquidHaskell:** Must ensure that functions terminate to safely verify certain properties.
- By default, LiquidHaskell proves termination of each recursive function.

Termination metric

A termination metric is a value that:

- Gets smaller with **every** recursive call.
- Cannot decrease forever (e.g., a non-negative integer).

LiquidHaskell Termination: Factorial

A simple termination metric is a **non-negative integer**.

```
1 {-@ fac :: n:Nat -> Nat / [n] @-}
2 fac :: Int -> Int
3 fac 0 = 1
4 fac n = n * fac (n - 1)
```

- `Nat`: integers greater than or equal to zero.
- `/ [n]`: use the argument `n` as the termination metric.
- The base case returns immediately.
- The recursive case decreases `n` by 1 each call.

LiquidHaskell Termination: Processing Batches

Consider a simplified **batch worker** that counts completed jobs.

- `b`: batches remaining, including the current batch.
- `j`: jobs remaining in the current batch.
- Completing a job decreases `j`, while `b` stays unchanged.
- Moving to the next batch decreases `b` and resets `j` to 3.

Two batches of three jobs

$(2,3) \to (2,2) \to (2,1) \to (2,0) \to (1,3) \to \cdots \to (1,0)$.

Neither argument strictly decreases at every call. We supply a **custom termination metric**: the pair `[b, j]`.

LiquidHaskell Termination: Lexicographic Ordering

The metric is written in the function's signature:

```
1 {-@ jobs :: b:Nat -> j:Nat -> Nat / [b, j] @-}
```

`/ [b, j]` uses **lexicographic ordering**: compare `b` first, and compare `j` only when `b` is unchanged.

- Completing a job, $(b, j) \to (b, j-1)$: `b` stays equal, `j` decreases.
- Next batch, $(b, 0) \to (b-1, 3)$: `b` decreases, so `j` may grow.
- Both components stay non-negative, so the pair cannot decrease forever.

Functional Correctness Invariants

LiquidHaskell Functional Correctness

Beyond avoiding errors, refinements can express **application-specific correctness rules**.

The paper's example: **red-black trees**, a balanced binary search tree structure.

```
1 data Col    = R | B
2 data Tree a = Leaf
3             | Node Col a (Tree a) (Tree a)
```

- `Col`: the color; `a`: the key; followed by the left and right subtrees.

LiquidHaskell Red-Black Trees: Three Invariants

A tree is only a valid Red-Black tree if:

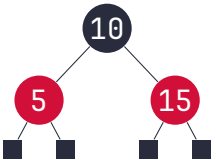
- **Order:** all keys in the left subtree are smaller than the node's key and all keys in the right subtree are larger.
- **Color:** a red node has only black children. Empty leaves count as black.
- **Height:** from any node, every path to a leaf contains the same number of black nodes.

Main idea

Describe these properties separately, then combine them without changing the underlying tree structure.

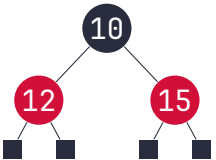
LiquidHaskell Red-Black Trees: Valid and Invalid

Valid



$5 < 10 < 15$.
Color and height are valid too.

Invalid



Order fails: $12 > 10$.

Black squares represent empty leaves.

LiquidHaskell Red-Black Trees: Checking Color

```
1 measure isB :: Tree a -> Prop
2 isB (Node c x l r) = c == B
3 isB (Leaf) = true
4
5 measure isRB :: Tree a -> Prop
6 isRB (Leaf) = true
7 isRB (Node c x l r) = isRB l && isRB r &&
8                       c == R => (isB l && isB r)
```

- `isB`: the root is black; empty leaves count as black.
- `isRB`: red nodes have black children, recursively.

Paper notation: from here on, specifications are shown without the `{-@ @-}` comment wrappers.

LiquidHaskell Red-Black Trees: Checking Height

```
1 measure bh :: Tree a -> Int
2 bh (Leaf) = 0
3 bh (Node c x l r) = bh l + if c == R then 0 else 1
4
5 measure isBal :: Tree a -> Prop
6 isBal (Leaf) = true
7 isBal (Node c x l r) = bh l == bh r && isBal l && isBal r
```

- `bh`: count black nodes along the leftmost path.
- `isBal`: require equal black heights, recursively.

LiquidHaskell Red-Black Trees: Checking Order

Ordering relates each node's key to **every key below it**, so it is built into the tree type itself.

- The tree type takes two **rules** as parameters: one for keys in the left subtree, one for the right.
- Each rule compares a node's key k with a key v below it.
- `OTree` sets the left rule to $v < k$ and the right rule to $v > k$.

Result

Every value of type `OTree` is a binary search tree.

LiquidHaskell Red-Black Trees: Combining the Rules

Combining ordering, color, and black-height balance:

```
1 type RBT a = {v:OTree a | isRB v && isBal v}
```

- `OTree`: keys follow binary-search ordering.
- `isRB`: red nodes have black children.
- `isBal`: all paths have equal black heights.

With these invariants and suitable function signatures, LiquidHaskell verifies the library's **insertion, deletion, and rebalancing** procedures.

Evaluation

LiquidHaskell Evaluation: Benchmarks

Library	Verified properties
GHC.List, Data.List	size properties of list operations
Data.Set.Splay	termination, well-ordered splay trees
Data.Map.Base	termination, binary-search ordered trees
HsColour	totality of all functions
XMonad	window uniqueness, some QuickCheck properties
ByteString	termination, memory safety, correctness
Text	pointer safety, correctness (found a bug)
Vector-Algorithms	safe vector accessing, indexing and slicing

LiquidHaskell Evaluation: Results

Module	LOC	Spec lines	Time (s)
GHC.List	309	38	15
Data.List	504	26	11
Data.Map.Base	1396	173	174
Data.Set.Splay	149	37	27
HsColour	1047	40	196
XMonad.StackSet	256	213	27
ByteString	3505	465	294
Text	3128	717	499
Vector-Algorithms	1218	266	89
Total	11512	1975	1336

LiquidHaskell Evaluation: What the Numbers Show

- **11,512** lines of code in 56 modules, checked with **1,975** lines of specifications.
- Hints (annotations and qualifiers) add only **3.5%** of the code size.

Performance

The full suite takes about **22 minutes**, but checking a handful of functions in isolation is fast enough for **interactive use**.

LiquidHaskell Limitations

The case studies also showed limitations of LiquidHaskell.

- **Ghost parameters:** values needed only for the proof must still be passed as extra arguments.
- **Fixed-width numbers:** integers are treated as unbounded and floats as exact reals, so overflow and rounding are not captured.

The paper lists several **further limitations** as well.

Workaround

In most cases, the code could be **altered slightly** to make verification possible.

References

- Niki Vazou, Eric L. Seidel, Ranjit Jhala. *LiquidHaskell: Experience with Refinement Types in the Real World*. Haskell Symposium, 2014.
- LiquidHaskell documentation: https://ucsd-progsys.github.io/liquidhaskell
- Satisfiability modulo theories: https://en.wikipedia.org/wiki/Satisfiability_modulo_theories